Welcome to CSC 340-02

Programming Methodology

Instructor: Joseph Hui
Email: jhui@sfsu.edu

Overview

- General Course Info
- Prerequisites
- Logistics
- Grading
- Class Policies
- Course Description

General Info

Instructor: Joseph Hui (jhui@sfsu.edu)

Office Hours: Scheduled zoom (by appointment-only via email)

Class Website: https://canvas.sfsu.edu

Class Syllabus: on Canvas

Class Schedule: Thursday 4:00pm – 6:45pm

Thanks:

Slides from this course are composites from Duc Ta, Hui Yang (SFSU), Paul Kube (UCSD), and textbook - Frank Carrano and Tim Henry (URI)

Course Prerequisites

- CSC 220 - Data Structures
- CSC 230 - Discrete Mathematical Structures for CS

- Both with grade C or better (no exceptions)
- Concurrent enrollment is not allowed

- CSC 340 assumes you know **Java** and **Data Structures**!

Required skills for this class

- Object oriented design and programming
- Programming in Java or C++
  - Flow control, looping, variables, file I/O, and arrays
- Fundamental Data Structures and algorithms
  - Stacks, queues
  - Tables
  - Sets
  - Maps
  - Hash Tables
  - Basic sorting and searching algorithms
- How to compute the sum of an arithmetic series
- Recursively define and solve a problem

Logistics

- Almost all communications will be through Canvas
- Make sure all of you have access to this course on Canvas. Course material, syllabus, homework submission, slides, will be posted there
- Briefly introduce yourself to your fellow classmates (everyone should post)
- Read the course syllabus

Grading

- Milestone #1 (10%) - IDE Installation and C++ DLL
- Milestone #2 (10%) - HashTable
- Milestone #3 (10%) - Cache Manager
- Milestone #4 (10%) - Binary Search Tree
- Milestone #5 (10%) - Cache Manager with BST

- Midterm I Exam: 25%
- Midterm II Exam: 25%
- Final Exam: 25% - top two of the three count

- Extra Credit 2%
  - Letter of Recommendation requirement

Exam Format

- Multiple Choice and T/F
  - 20 per exam
  - 1 point per question
  - Total 20 points

- Essay
  - 8 per exam
  - 5 points per question
  - Look specific phrases in answer
  - Total 40 points

Exam Notes

- Must be taken in class
  - Exceptions only for DPRC
- Government issued photo ID is required when turning in the exam
  - Student ID or a driver's license
  - Exam will not be accepted without an ID
  - Photo on your phone is not sufficient
- One cheat sheet is allowed
  - 8.5" x 11"
- No makeup exams will be given
  - If you miss a Midterm, then you must take the Final

Milestones

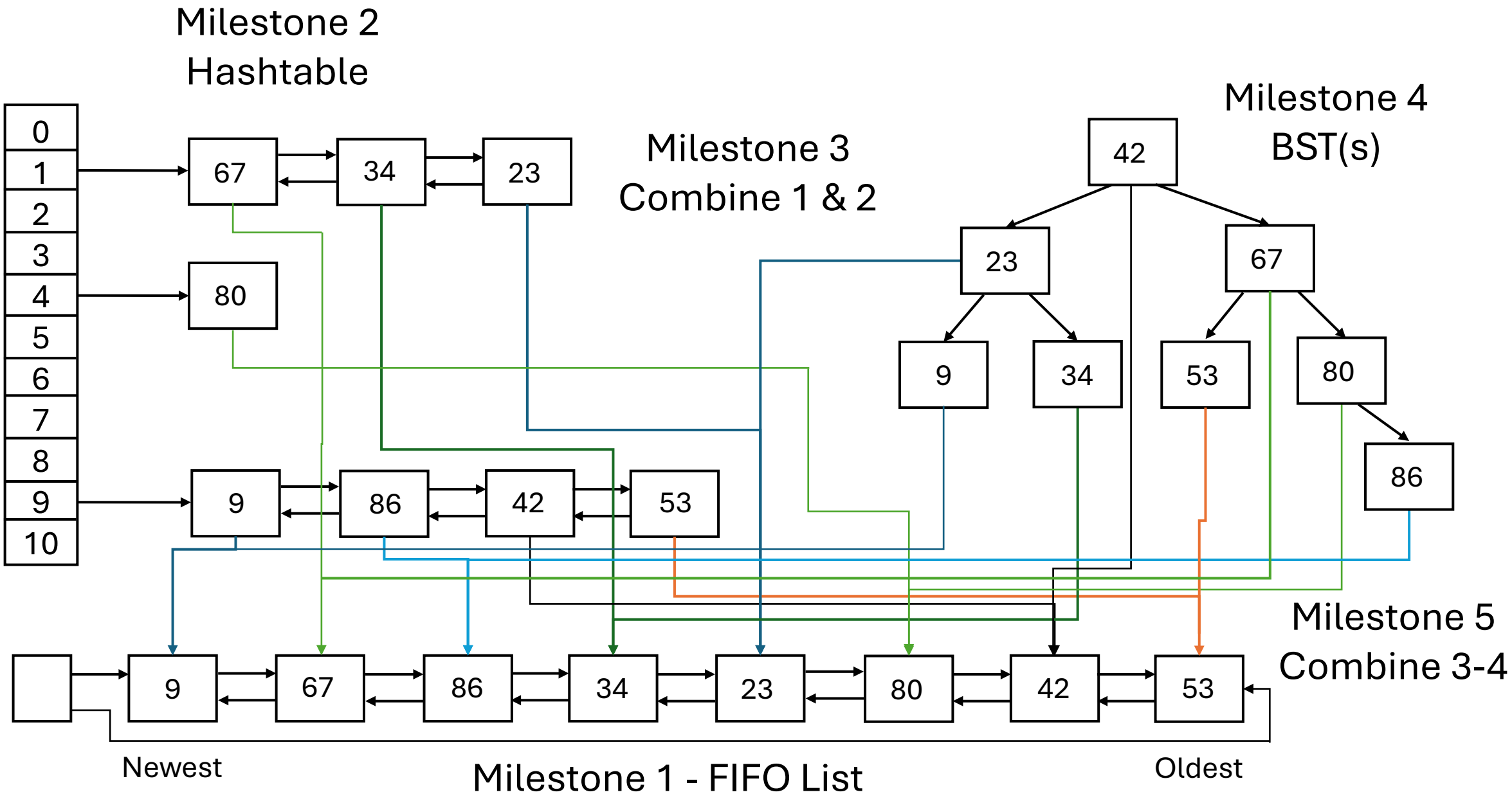
- 5 Milestones in this class
  - Subsequent Milestones depend on previous Milestones
- Each Milestone is worth 10% of final grade
  - Missing one will drop your grade by a letter (95% to 85% -> A to B)
- Late Milestones will be marked down 10% a day
  - After 7 days, a 0 will be given
  - Exceptions will need documentation (i.e., a doctor's note)
- Generative AI is allowed, but discouraged
  - Avoid using AI as a crutch
  - Software Engineering is a skill that improves with practice

Milestone Teams

- Milestone 1 needs to be done individually
  - Everyone needs to install a C++ IDE and write an implementation of a Doubly Linked List in C++

- Milestones 2-5 are team projects
  - Teams will be assigned
  - If your teammate does not communicate or drops the class, then you must complete the project on your own
  - No changing of teams is allowed
    - Changing teams will result in a 0 for the assignment
    - If you don’t respond to your teammate, you get a 0 (must document)

Milestone Overview – Cache Manager



Class Policies

- Cheating and plagiarism is taken very seriously
  - Read the course syllabus for more details
- If you have an emergency and cannot make it to class lecture, please let me know by email as soon as possible
- Consistently check the general channel for this course on Canvas
  - New announcements, grades, questions, sample homework, etc.

Expected Class Norm

- Respect each other
- Listen to each other
- Learn from mistakes
- Consciously cultivate joy in learning
- Actively participate during each lecture
- Offer the instructor and each other timely and constructive feedback/suggestions
- Grow together
- Anything else you would like to add to this list?

Class Overview

- CSC 340 is one of the first upper division classes for CSC
- This is a difficult class with a lot of ground to cover
  - Feedback from previous students is that I go very fast
- Much less hand holding, so you'll need to learn how to learn
  - Similar to how working in industry will be
- Expectation is to work on class homework 10-15 hours/week
  - Reading assignments are to be read BEFORE class (Appendix K, A, B)

Class Content

- Programming Methodology
  - What is Programming Methodology?
- Introduction to C++
  - Syntax, memory management, STL, C++ versus Java
- Advanced Data Structures
  - More coverage of Data Structures studied in CSC 220

Projects for this class

- Lectures will go fast, and missing lectures is not advised
- Going to discuss and work on industry/real world problems
- Use of Gen AI is discouraged
  - Generated code must be cited as such
- Each Project will build on the previous Project
  - Missing a Project will still need to be done before subsequent Projects

Syllabus – Weeks 1-7

Week	Date	Pre-Reading	Topic	Assignments
1	1/28	Appendix K, B	Syllabus, Course Policies Intro to Programming Methodology (PM)	
2	2/04	Appendix A, G, H, I	C++ versus Java C++ Basics	Milestone #1 Assigned
3	2/11	Chapter 1 Appendix C	PM: Problem Solving DS: Data Abstraction I (ADT)	
4	2/18	Interlude 1 Chapter 3	DS: Data Abstraction II C++: Classes, arrays, Memory Management I	Milestone #1 Due Milestone #2 Assigned
5	2/25	Interlude 2 Chapter 4	C++: Pointers, Templates	
6	3/04	Chapters 2, 10	C++: Looping, Complexity, Recursion Midterm I review	Milestone #2 Due
7	3/11		**Midterm 1**	

Syllabus – Weeks 8-17

Week	Date	Pre-Reading	Topic	Assignments
8	3/18	Chapter 18 Other readings	PM: Design Decisions (Choosing DS) DS: Dictionaries, Hashing	Milestone #3 Assigned
9	3/25		No Class (Spring Break)	
10	4/01	Interludes 2, 3, 4	C++: Inheritance and Polymorphism Revisit Milestones #2-5 Introduce Design Patterns	Milestone #3 Due Milestone #4 Assigned
11	4/08	Interlude 1.3, 7 Chapter 8.3,	C++: Templates, STL (BIG 5) Discuss Milestone #4	
12	4/15	Interludes 3, 4, 6	C++: Overloading, Exception Handling, and Smart Pointers, Memory Management II	
13	4/22	Chapters 15	Tree algorithms Midterm II Review	Milestone #4 Due
14	4/29		**Midterm II**	
15	5/06	Chapters 11, 16, 19	C++: Tree Implementations Sorting and Balanced Search Trees	Milestone #5 Assigned
16	5/13	Chapter 20	Final Review C++ and DS: Graphs Guest Lecturer	
17	5/20		Final Exam	Milestone #5 Due

Programming Methodology

"Programming Methodology is the way of **analyzing** a particular complex problem by **planning** the software development and **controlling** the development process."

- Current most popular methodology is Object Oriented
  - Create objects and their attributes (nouns) and methods (verbs)
  - Inheritance, Encapsulation, and Polymorphism

- Is AI a methodology?

Is AI a Methodology?

"AI won't take your job,
it's somebody using AI that will take your job."

Richard Baldwin

2023 World Economic Forum's Growth Summit

Development and Debugging Process

- The IDEAL Problem Solver
  - By John Bransford and Barry Stein

- I - Identify
  - Dev - Product Managers identify a problem (RUP – Inception phase)
  - Debug - Easy for debugging software (QA or user will find a bug) and look for potential future problems
- D - Define
  - Dev - Product Managers define the problem (RUP – Inception phase)
  - Debug - What use cases reproduce the error and what doesn't
- E - Explore
  - Dev - Architects negotiate functionality with Product Managers (RUP – Elaboration phase, iterative)
  - Debug - Find the problem area and explore alternative fixes
- A - Act on a plan
  - Dev - Architects and developers design and write the code (RUP – Construction, iterative)
  - Debug - Fix the bug
- L - Look at the effects
  - Dev - QA verifies that the new software works (RUP – Transition, iterative)
  - Debug - Verify the bug fix and add test case

Testing

Type	Description
Unit Testing	A programmatic test that tests the internal working of a unit of code, such as a method or a function. Generally written by the developer.
Minimal Acceptance Test (MATs)	A quick set of tests that verify a version of software meets the minimum quality level. Standard process is to run MATs before a code merge. Full Regression Testing can take days, so MATs are run to verify basic functionality.
Regression Testing	Ensures that existing features/functionality that used to work are not broken due to new code changes. Generally written by Quality Assurance.
System Testing	Complete end-to-end testing is done on the complete software to make sure the whole system works as expected.
Smoke Testing	A quick test performed to ensure that the software works at the most basic level and doesn't crash when it's started. Its name originates from the hardware testing where you just plug the device and see if smoke comes out.
Performance Testing	Ensures that the software performs according to the user's expectations by checking the response time and throughput under specific load and environment.
User-Acceptance Testing	Ensures the software meets the requirements of the clients or users. This is typically the last step before the software is live, i.e. it goes to production.
Stress Testing	Ensures that the performance of the software doesn't degrade when the load increases. In stress testing, the tester subjects the software under heavy loads, such as a high number of requests or stringent memory conditions to verify if it works well.
Usability Testing	Measures how usable the software is. This is typically performed with a sample set of end-users, who use the software and provide feedback on how easy or complicated it is to use the software.
Security Testing	Now more important than ever. Security testing tries to break a software's security checks, to gain access to confidential data. Security testing is crucial for web-based applications or any applications that involve money. Generally done by the Security Testing Group (fun job!)

https://www.interviewbit.com/software-testing-interview-questions/

Test- Driven Development (TDD)

1. Identify a new feature
2. Write a unit test for that feature (note test is written before the code)
3. Run the test. It should fail.
4. Write code that passes test.
   - No self documenting code!
5. Optional: refactor code to make it faster, simpler, more supportable, etc.

Test Question

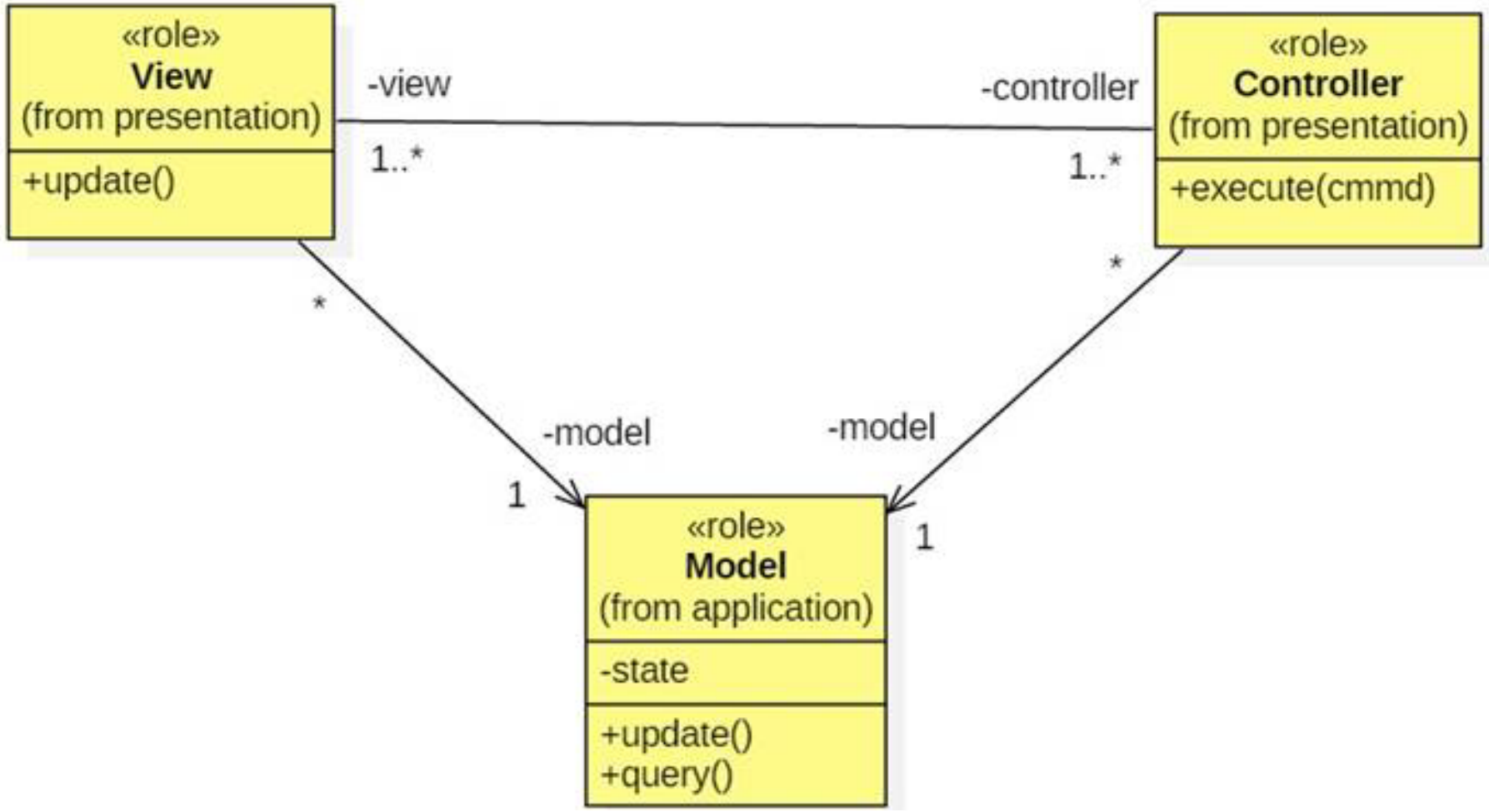
- What type of testing have you utilized in your CSC 220/CSC 221 course?

Effective interfaces

- User Interface (UI)
  - Easy to use
  - Useful error message

- Application Programming Interface (API)
  - Easy to use
    - Name methods and parameters effectively
    - Simple parameter list
  - Exception handling

User Interface – modern UI's use Model View Controller (MVC)



Professionally written code

- Effective and efficient
  - Does what it is supposed to do
  - Does not consume more CPU or memory than needed
  - Performant
  - Stable/Reliable

- Maintainability
  - Human understandable
  - Take advantage of Libraries and Frameworks
  - Documentation, Documentation, and more Documentation
  - Testing

- Modular
  - Break large problems into smaller problems
  - Larger methods are generally harder to reuse

Programming Style

- Writing easy to read and effective programs is a skill

- Strive to be always write clear and understandable code
  - 90% of the cost for industry code is for maintenance
  - Clear and understandable code is maintainable code

- Follow corporate coding standards
  - Good names for variables and methods
  - Always create File and Method comment headers
  - Comment the code for the next person
  - Liberal use of whitespace

Experience counts

- The more you code, the better you will get at it
- Practice, practice and more practice writing code
- Spend more time on important code
  - Good, Fast, and Cheap
  - Write code fast and cheap. Once determine which is most used, then improve
- Listen and learn from those before you

Generative AI as a Crutch

- Dependence on Generative AI can be detrimental to your skills
- ~80% of generated code is good (this will improve)
- How will you determine which 20% is not so good?
- Experience is needed to determine good from not so good
- Experience comes from practice and more practice

Death by a Thousand Paper Cuts

- Working on code is difficult

- Little bothersome issues add up
  - Your own (unmatching) coding style
  - Use your name or pets' name for variables
  - Multiple lines of code in a single line
  - Hard to understand custom code (i.e., non-standard):
    - Algorithms
    - Data structures

- Use Design Patterns

Closing Words on Programming Methodology

"Any fool can write code that a computer can understand.

Good programmers write code that humans can understand."

Martin Fowler

Author of multiple OO and UML texts

IEEE Top Programming Languages

- Graphs from googling “IEEE popular programming languages”
  - IEEE stands for “Institute of Electrical and Electronics Engineers”
- The “Spectrum” ranking is weighted towards the profile of the typical IEEE member
- The “Trending” ranking seeks to spot languages that are in the zeitgeist
- The “Jobs” ranking measures what employers are looking for.

Programming Languages by Spectrum

IEEE Spectrum | The Top Programming Languages 2024

Type to search

SHARE THIS STORY

TAGS

TOP PROGRAMMING LANGUAG…

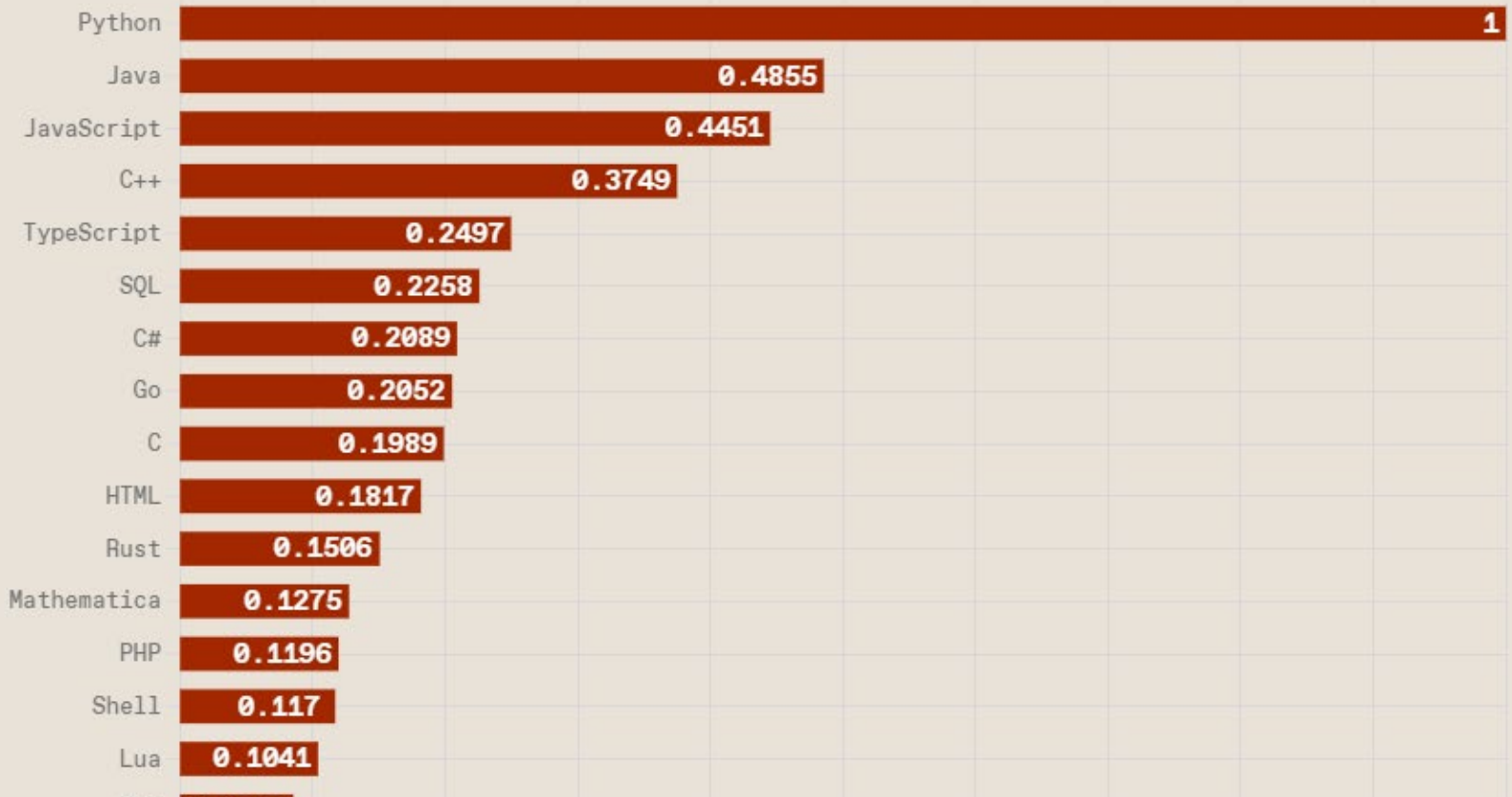
PYTHON | SQL | RUST

TYPESCRIPT

Top Programming Languages 2024

Click a button to see a differently weighted ranking

Spectrum | Trending | Jobs



Programming Languages by Trending

IEEE Spectrum / The Top Programming Languages 2024

Type to search

SHARE THIS STORY

TAGS

TOP PROGRAMMING LANGUAG…

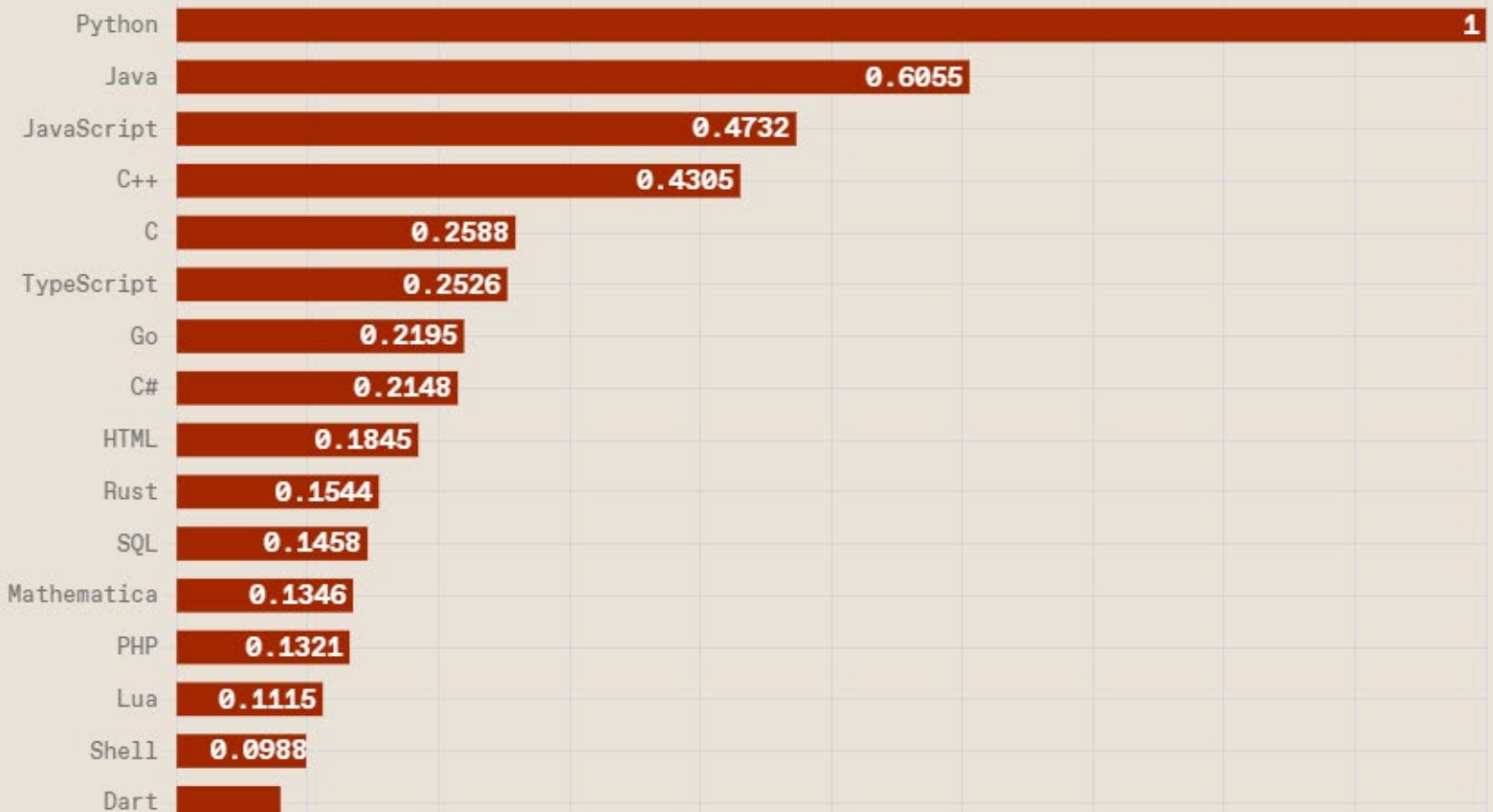
PYTHON | SQL | RUST

TYPESCRIPT

Top Programming Languages 2024

Click a button to see a differently weighted ranking

Spectrum | Trending | Jobs



Programming Languages by Jobs

IEEE Spectrum / The Top Programming Languages 2024

Type to search

SHARE THIS STORY

TAGS

TOP PROGRAMMING LANGUAG…

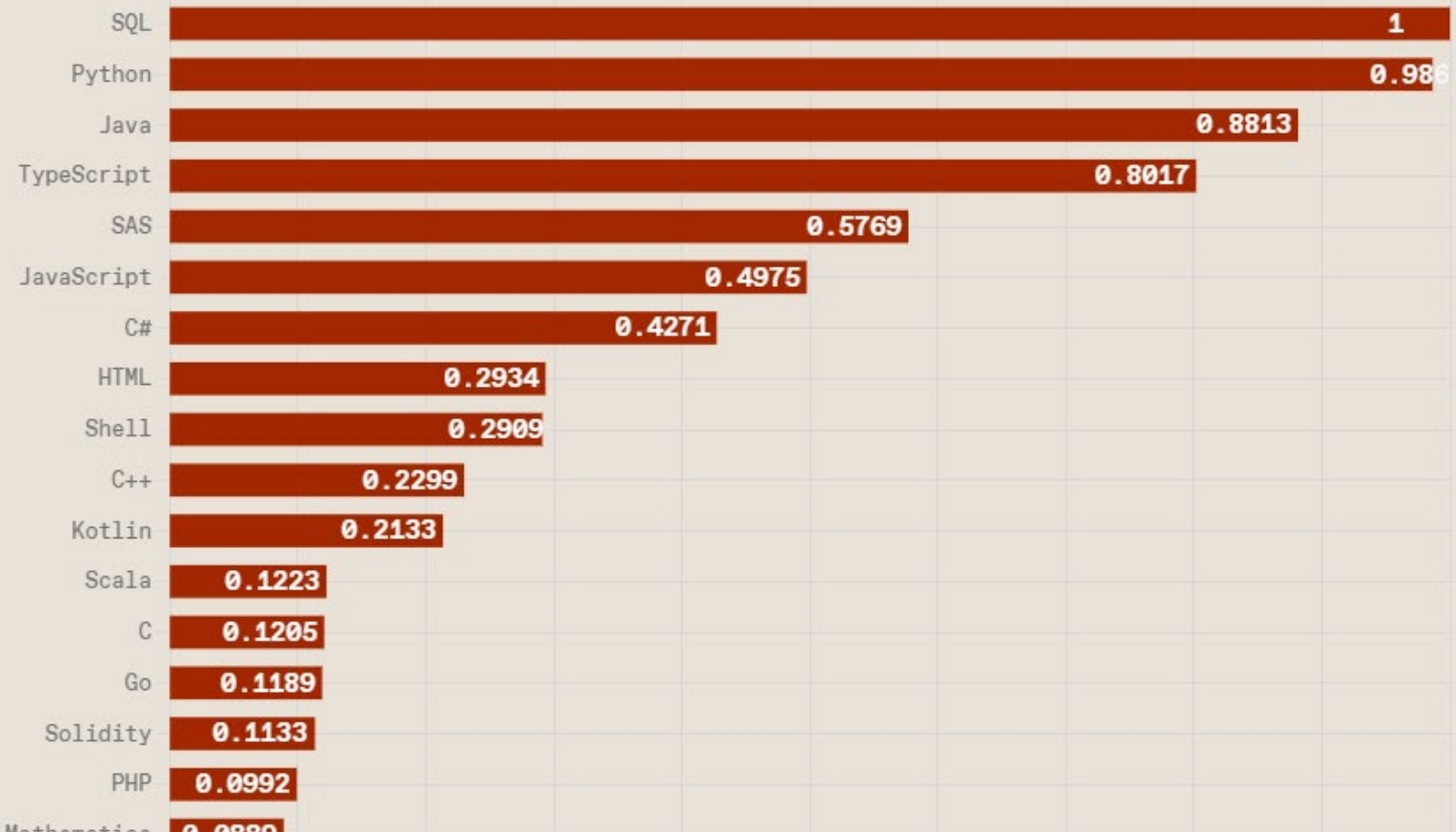
PYTHON | SQL | RUST

TYPESCRIPT

Top Programming Languages 2024

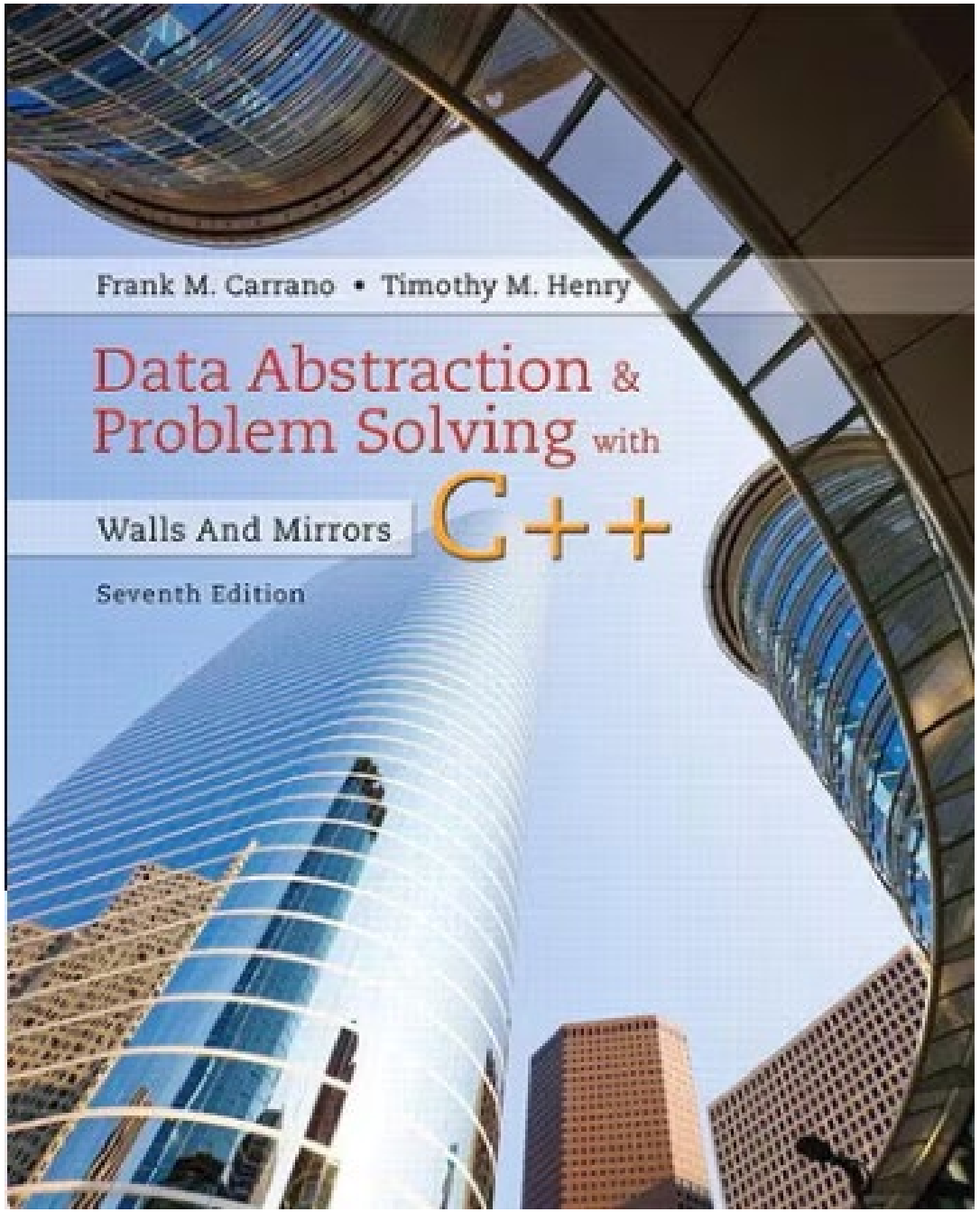
Click a button to see a differently weighted ranking

Spectrum | Trending | Jobs



Introduction to C++

Data Abstraction & Problem Solving with C++: Walls and Mirrors, 7th Edition



- Syntax
- Memory management
- STL
- Similarities/Differences from Java

Introduction to C++

- C++ is an object-oriented language based on C
- C++ was developed by Bjorne Stroustrap at Bell Labs (1980’s)
- CSC 340 doesn’t assume experience in C or C++
- CSC 340 does assume experience with Java

C++ Syntax

1. Enables input and output ----------> `#include <iostream>`
2. Opens the standard namespace ----> `using namespace std;`
3. Begins the function **main** ----------> `int main()`
4. A comment ----------> `// Computes the volume of a sphere of a given radius.`
5. Begins body of function ----------> `{`
6. Defines a constant ----------> `const double PI = 3.14159;`
7. Declares a variable ----------> `double radius;`

8. Displays a prompt to the user ------> `cout << "Enter the radius of the sphere: ";`
9. Reads **radius** ----------> `cin  >> radius;`

10. Declares and computes **volume** ---> `double volume = 4 * PI * radius * radius * radius / 3;`
11. Displays results ----------> `cout << "The volume of a sphere of radius "`
12. Statement continues ----------> `<< radius << " inches is " << volume`
13. Statement continues ----------> `<< " cubic inches.\n";`
14. Normal program termination ------> `return 0;`
15. Ends body of function ----------> `}  // end program`

C++ Memory Management

- In Java, all objects are created through `new`, and the memory will be reclaimed with the garbage collector when no longer referenced
- In C++, objects can be created via `new`, but the memory is not automatically reclaimed
- In C++, if an object doesn’t have a pointer to it, it is a memory leak because it can’t be reclaimed.
- Memory leaks are very bad and eventually will bring the software down

C++ and Standard Template Library (STL)

- Functionally, similar to Java generics
- In C++, best to explicitly include only what is needed

```
#include <list>
```

- In C, code is in .c files and interfaces/declarations in .h (header files)
  - In C++, header files can contain implementation
  - .hpp is a convention to differentiate from C headers
- However, template code is contained in .hpp files
  - injected at compile time

C++ vs Java

- Memory management
- STL vs generics
- Java initializes primitive member variables (C++ does not)
- In C++, pointers are different than references

Advanced Data Structures

- Review Data Structures covered in CSC 220
  - If you have not taken a Data Structures class, please take CSC 220
- Implementation
  - Implementation in C++
  - Using Data Structures with STL
- Go into more depth for when to use a particular Data Structure
  - Complexity of algorithms for operations

Data Structure Overview

- Provides approach to store, organize and retrieve data efficiently
- Basic types: queue, stack, linked list
- Advanced types: tree, hash table, graph
- Certain data structures lend themselves to certain applications
  - Database – b-trees
  - Compiler – hash tables, stack
- Data structure + algorithms = programs

Implementation in C++

- Review implementation in java
- Assignments are to implement them in C++
- Not from scratch (skeleton/example code will be provided)
- Template implementation

When to use a particular Data Structure

- Go over scenarios and decide which Data Structure to use

- Scenarios will be from Job Interview Questions
  - Almost all interviews for software development have Data Structure questions
  - My personal favorites, as well as interview questions on web

- The one skill I want you to develop in this class, is the ability to analyze a problem and determine which Data Structure(s) to use to solve it.
  - Good for Job Interviews
  - But, more importantly, it’s good to know for your career

Industry information

- Programming versus Software Engineering
- Writing and Presenting to communicate (i.e., Architectural/Design docs)
- Comments as code
- Peer review of code
- Unit Tests to verify functionality
- Earlier you catch an error, the cheaper it is (compile, QA, Regression, Production)

Your interview experience: full-time roles

Coding Sample & Survey

Candidates will have 90 minutes to complete this coding sample

1–2 Phone Interviews

45 minute technical phone interviews with engineers

5 Onsite Interviews

4 -45 minute technical onsite interviews and 1 Google interview with a lunch break in the middle of the day

Team Review

Offer

Full time offers can be negotiated

Can take about 2 months from start to finish

HR/Recruiter Interview

HR will ask “soft skill” questions

- Tell me about what our company does?
- Give an example of a difficult subordinate/peer/supervisor and how you handled it
- Give an example of a difficult problem and how you solved it
- Give an example of showing that you are a team player

Technical Interview Content

Coding

Java, C, C++, JavaScript,
Python, Go
Methods
Classes

Algorithms

Sorting
Searching
Traversals
Big-O analysis
Recursion

Data Structures

Lists
Stacks/queues
Trees/graphs
Sets
Hashes/maps
Heaps

Books For Software Engineers

- The Art of Computer Programming - Fundamental Algorithms (Knuth)
  - Covers most topics for a BS in Computer Science (3 x ~$60)
- Unix Network Programming (Stevens)
  - First 7 chapters give a great overview of Unix and TCP/IP (~$45)
- Design Patterns (Gamma, Helm, Johnson, Vlissides, aka Gang of Four)
  - Concepts for advanced Object Oriented programming (~$45)
- Database Tuning – A Principled Approach (Shasha and Bonnet)
  - Basics of database tuning (~$65)
- Introduction to Parallel Programming (Pacheco and Malensek)
  - Most modern programs require multi-threading (~$65)